

Rick Sarkar
Professor Lin
23 February 2026

CS 3220 Lab 2 Writeup

Baseline:

My original branch predictor achieves a baseline accuracy of 32.29%.

I used a `$display` command in the FETCH stage to see the exact counter values for the total number of branch/jump instructions in the towers test file, as well as the number of correct predictions. The results were the following: 320 correct out of 991 total branches/jumps.

```
FE - Total: 000003df, Correct: 00000140
```

While this exceeds the 30% accuracy benchmark, I wanted to see how altering the design would affect (and possibly improve) the baseline accuracy.

Design Changes:

I experimented with three design changes and measured how well these altered branch predictors performed compared to the baseline above.

1. Hashing Update

One limitation of the PHT is how the index is computed by XORing a portion of the PC with the contents of the GHR. When two different combinations hash to the same index, they can overwrite each other's target addresses and cause more frequent mispredictions. The goal of altering the hashing function in this design change is to observe if this hash function will have fewer collisions and therefore a higher accuracy.

To do this, I altered the range of bits of the GHR and PC are XOR'd together, by shifting the range being XOR'd to the left by 15 bits. This boosted the accuracy to **37.94%**. It seems that the new hashing function did indeed result in lower collisions in the baseline, and an overall increase of 5.65 in branch prediction accuracy over the baseline.

```
FE - Total: 000003df, Correct: 00000178
```

2. Altering PHT Size

Another idea is to alter the size of the PHT to store fewer values. At first, I found this unintuitive since we would logically have more conflicts with our hash function. However, a smaller PHT will have fewer “invalid” slots on startup, meaning there will be fewer cold misses at the start of the program. For a small or recursive program, this can mean the PHT is quickly populated with accurate targets and doesn’t waste empty entries. In this case, shrinking the PHT indeed improves branch prediction accuracy, especially in combination with the update to the hash function. This resulted in an accuracy of **39.56%**.

```
Total: 000003df, Correct: 00000188
```

3. The final design change considered was removing the tag comparison, and preferring an optimistic BTB policy.

I was interested in the potential impact on aliasing/collisions for the BTB. The result was unfortunately a reduction in accuracy to 31.58%. This was somewhat expected, but it proves the tag comparison is useful for prediction accuracy. However, since the drop in accuracy was not huge, we can possibly conclude that the functionality of the predictor is not wholly dependent on this comparison.

```
FE – Total: 000003df, Correct: 00000139
```

In conclusion, two of the three design changes achieved an improvement of over 5% accuracy over the baseline. These changes to the hashing function and PHT size proved to be valuable changes to the accuracy of my branch predictor.